TECH ELLIPTICA

PLAYWRIGHT

Commands Cheat Booklet

VAIBHAV SINGH

Playwright Commands Cheat Booklet

Installation & run

Command	Description
`npm init playwright@latest`	New Playwright project (interactive)
`npm install -D @playwright/test`	Add Playwright to existing project
`npx playwright install`	Install browsers
`npx playwright test`	Run all tests
`npx playwright test <file>`	Run one file
`npx playwright test -g "login"`	Run tests with title matching "login"
`npx playwright test --headed`	Run in headed (visible) browser
`npx playwright test --debug`	Run in debug mode
`npx playwright test --ui`	Open Playwright UI mode

Run options & report

Command	Description
`npx playwright test --project=chromium`	Run only Chromium
`npx playwright test --workers=1`	Run tests one at a time
`npx playwright test --retries=2`	Retry failed tests twice
`npx playwright test --reporter=html`	Generate HTML report
`npx playwright show-report`	Open last HTML report
`npx playwright codegen <url>`	Record tests in browser
`npx playwright codegen --save-storage=auth.json`	Save storage (e.g. login)
`npx playwright test --trace=on`	Record trace for debugging

Locators (getBy)

API	Example
`getByRole`	`page.getByRole('button', { name: 'Submit' })`
`getByText`	`page.getByText('Welcome')`
`getByLabel`	`page.getByLabel('Email')`
`getByPlaceholder`	`page.getByPlaceholder('Enter name')`
`getByAltText`	`page.getByAltText('Logo')`
`getByTitle`	`page.getByTitle('Close')`
`getByTestId`	`page.getByTestId('submit-btn')`
`locator` (CSS)	`page.locator('button.primary')`
`locator` (XPath)	`page.locator('//button[@type="submit"]')`

Actions

Method	Example
`click()`	`page.getByRole('button').click()`
`dblclick()`	`locator.dblclick()`
`fill()`	`page.getByLabel('Name').fill('John')`
`press()`	`page.getByRole('textbox').press('Enter')`
`check()` / `uncheck()`	`page.getByLabel('Accept').check()`
`selectOption()`	`page.getByLabel('Country').selectOption('IN')`
`setInputFiles()`	`page.getByLabel('File').setInputFiles('file.pdf')`
`hover()`	`locator.hover()`

Assertions

Assertion	Example
`toHaveTitle`	`await expect(page).toHaveTitle(/Dashboard/)`
`toHaveURL`	`await expect(page).toHaveURL(/login/)`
`toBeVisible`	`await expect(locator).toBeVisible()`
`toHaveText` / `toContainText`	`await expect(locator).toContainText('Welcome')`
`toHaveValue`	`await expect(input).toHaveValue('john')`
`toBeChecked`	`await expect(checkbox).toBeChecked()`
`toHaveCount`	`await expect(list).toHaveCount(5)`
Soft	`await expect.soft(locator).toBeVisible()`
Not	`await expect(locator).not.toBeVisible()`

Page & navigation

Method	Example
`goto()`	`await page.goto('https://example.com')`
`goBack()` / `goForward()` / `reload()`	Navigation
`waitForURL()`	`await page.waitForURL('**/dashboard')`
`waitForSelector()`	`await page.waitForSelector('.loaded')`
`setDefaultTimeout()`	`page.setDefaultTimeout(10000)`
`setDefaultNavigationTimeout()`	`page.setDefaultNavigationTimeout(15000)`

Config (playwright.config)

```
module.exports = {
  testDir: './tests',
  timeout: 30000,
  retries: 1,
  workers: 4,
  use: {
    baseURL: 'https://example.com',
    headless: true,
    viewport: { width: 1280, height: 720 },
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
};
```

Fixtures & hooks

Usage	Example
Test	`test('title', async ({ page }) => { ... })`
Describe	`test.describe('Login', () => { ... })`
Skip / Only	`test.skip(...)` / `test.only(...)`
Before each	`test.beforeEach(async ({ page }) => { ... })`
After each	`test.afterEach(async ({ page }) => { ... })`
Before all / After all	`test.beforeAll(...)` / `test.afterAll(...)`

Examples

Login flow:

```
const { test, expect } = require('@playwright/test');
test('login', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@test.com');
  await page.getByLabel('Password').fill('secret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page).toHaveURL(/dashboard/);
});
```

Assert list count:

```
const rows = page.locator('table tbody tr');
await expect(rows).toHaveCount(10);
await expect(rows.first()).toContainText('Alice');
```

MCQ

Q1. What is the default timeout for a single assertion in Playwright (e.g. `expect(locator).toBeVisible()`)?

A) 1 second

B) 5 seconds

C) 0 (no wait)

D) Same as test timeout (default 30s for the assertion timeout)

Q2. When using `page.locator('button').first()`, when does Playwright resolve which element is "first"?

A) At the time of the call
B) At the time of the action/assertion (auto-waiting and re-query if needed)
C) Only once at page load
D) Never; it throws

Q3. What is the difference between `expect(locator).toHaveText('exact')` and `expect(locator).toContainText('exact')`?

A) No difference
B) toHaveText matches the full text (or regex); toContainText matches substring
C) toContainText is deprecated
D) toHaveText is for soft assertions only

Q4. How do you run tests in a specific order (e.g. one after another) in Playwright?

A) Use `test.describe.serial()` or run with `--workers=1` for that file

B) Use test.only in sequence

C) Order is always random

D) Set order in config with array of files

Q5. What does `test.info().attach()` typically do?

A) Attach a file (e.g. screenshot, trace) to the test report

B) Attach a tag

C) Attach to another test

D) Nothing; it is not a real API

Q6. When using `page.route()`, what does `route.fulfill({ status: 200, body: '...' })` do?

A) Proceeds to the real server
B) Responds to the request with the given status and body (mocking)
C) Aborts the request
D) Retries the request

Q7. What is the scope of `test.beforeEach` when used inside `test.describe('Login', () => { ... })`?

A) Runs before every test in the entire project
B) Runs before every test in that describe block only
C) Runs once before the describe
D) Same as afterEach

Q8. How do you run only tests that have a certain annotation (e.g. `test.annotation`) in Playwright?

A) Use `--grep` or project dependency / config

B) Use @tag in test name

C) Use test.only

D) Playwright does not support annotations for filtering

Q9. What happens if you call `await page.goto('...')` and the page never fires `load` (e.g. slow network)?

A) Test passes immediately

B) Test waits until default navigation timeout (or your configured one), then fails

C) Test waits forever

D) Playwright skips the goto

Q10. In `playwright.config.js`, what does `use: { trace: 'on-first-retry' }` mean?

A) Trace is always recorded
B) Trace is recorded only when a test is retried (e.g. after first failure)
C) Trace is recorded only on first run
D) Trace is disabled

Q11. What is the purpose of `test.describe.configure({ mode: 'serial' })`?

A) Run all tests in that describe block one after another in the same worker
B) Run tests in alphabetical order
C) Run only one test
D) Run in parallel with other describes

Q12. How do you get the `page` object inside a custom fixture that extends the built-in `page`?

A) You cannot
B) It is the first argument: `async ({ page }, use) => { ... }`
C) Via `test.info().page`
D) Via global variable

Q13. What does `locator.waitFor({ state: 'attached' })` do?

A) Waits until the element is visible
B) Waits until the element is in the DOM (attached), not necessarily visible
C) Waits until the element is hidden
D) Same as toBeVisible()

Q14. When using `expect.soft()`, what happens after the first soft failure?

A) Test stops immediately
B) Test continues; all soft assertions run; then test fails if any failed
C) Test is skipped
D) Only the first soft assertion is reported

Q15. What is the recommended way to share login state across tests (e.g. avoid logging in every time)?

A) Log in in every test
B) Use storageState: save auth state once (e.g. via context.storageState()), then use it in config or project as storageState path
C) Use globalSetup only
D) Use beforeAll and share page (not recommended; use storageState instead)

MCQ Answers

Q1. B (assertion timeout default 5s)

Q2. B

Q3. B

Q4. A

Q5. A

Q6. B

Q7. B

Q8. A

Q9. B

Q10. B

Q11. A

Q12. B

Q13. B

Q14. B

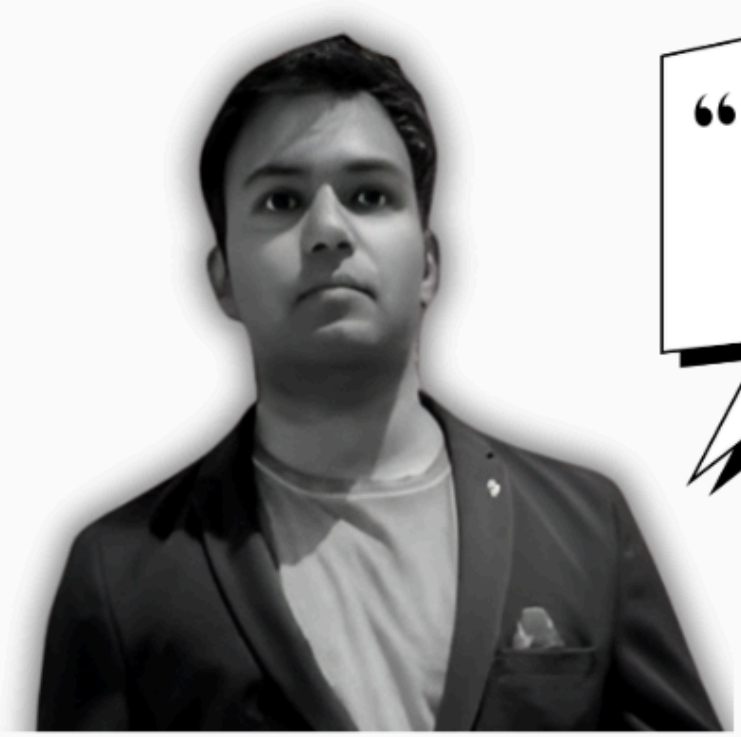
Q15. B

Assignments

#	Assignment	What to do
1	Title assertion	Write a test that goes to https://example.com and asserts the page title contains "Example".
2	getByRole and click	Use `getByRole` to find a link and click it, then assert the URL changed.
3	baseURL	In config, set `baseURL` and use `page.goto('/about')` in a test.
4	API mock	Use `page.route()` to mock a GET request and return custom JSON; write a test that asserts the mocked data appears.
5	Soft assertions	Create a test with `expect.soft()` for two assertions; make the first fail and the second pass; confirm the test fails and both results are reported.

POWERED BY

VAIBHAV SINGH



"LIVE A LIFE YOU REMEMBER...!"

Join
Book Portal

HTTPS://WWW.LINKEDIN.COM/IN/VAIBHAV-SINGH-BY-PUBLICIS/

TECH ELLIPTICA

https://www.techelliptica.com